UADE

SISTEMA DE GESTIÓN DE **CLÍNICA ODONTOLÓGICA**

Paradigma Orientado a Objetos

Presentado por:
Aranda, Fabian
Zorro, Brayan
Soto, Christian
Pabón, Gabriel

ÍNDICE

CONTENIDO DEL INFORME

Introducción 1

Diseño del Dominio *2*

a. Principio DRY *2.1*

b. Entidades Base *2.2*

c. Entidades Adicionales *2.3*

Relaciones y Cardinalidad 3

Comportamiento y Encapsulamiento 4

Conclusión de la Entrega 5

INTRODUCCIÓN

El presente documento detalla las decisiones arquitectónicas y de diseño orientadas a objetos correspondientes a la Entrega 1 del sistema "Sonrisa Feliz". Esta estructura fundamental servirá como cimiento lógico y base de datos temporal en memoria para la posterior implementación de las capas de Servicios y Repositorios en las siguientes iteraciones.

DISEÑO DEL DOMINIO

Partiendo de los requisitos básicos, el equipo decidió implementar las siguientes entidades estructurales, aplicando buenas prácticas de POO:

APLICACIÓN DEL PRINCIPIO DRY (DON'T REPEAT YOURSELF)

Se identificó una fuerte redundancia de atributos entre los actores del sistema. Por lo tanto, se diseñó una Superclase Abstracta ***Persona***.

- **Atributos base:** id, nombre, apellido, email.
- **Comportamiento común:** getNombreCompleto().
- **Justificación:** Centraliza la información básica. Las entidades *Paciente*, *Odontologo* y *Recepcionista* heredan de esta superclase. Es abstracta dado que el dominio no requiere la instanciación de "Personas" sin un rol definido.

ENTIDADES DEL DOMINIO BASE

- **Paciente:** Extiende de *Persona*. Añade dni, fechaIngreso, una composición con *Domicilio* y una lista con su historial de *Turnos*.
- **Odontologo:** Extiende de *Persona*. Añade matricula y una composición con *Agenda*.
- **Turno y EstadoTurno:** La entidad central que vincula a un paciente con un profesional en una fecha y hora específica. Su estado se controla mediante un Enum (PENDIENTE, CONFIRMADO, CANCELADO, COMPLETADO) para asegurar la consistencia de los datos.

ENTIDADES ADICIONALES (DECISIONES DE ESCALABILIDAD)

Para evitar clases "Dios" y distribuir responsabilidades, se incorporaron:

- **Agenda:** Para evitar que la clase *Odontologo* tenga demasiadas responsabilidades, se extrajo su lista de turnos hacia una clase propia. La *Agenda* es la encargada exclusiva de guardar su colección de turnos y revisar de forma interna si un horario está disponible, tal como ocurre en un consultorio real.
- **Recepcionista:** Aunque los requisitos la mencionan como un Actor, se modeló como entidad para esta entrega. Esto permite simular su interacción real en la clase Main, delegándole temporalmente la responsabilidad de instanciar pacientes y gestionar turnos.

RELACIONES Y CARDINALIDAD

1. **Herencia (Generalización):** Las clases *Paciente*, *Odontologo* y *Recepcionista* heredan de la superclase abstracta *Persona*, compartiendo así los atributos básicos y el comportamiento común (relación "Es un/a")
2. **Composición (1 a 1):** El *Paciente* contiene a su *Domicilio* y el *Odontologo* a su *Agenda*. Estos objetos secundarios no tienen sentido en nuestro sistema sin su clase contenedora, atando su ciclo de vida al de la entidad principal.
3. **Asociación (1 a Muchos):** La *Agenda* tiene una relación de uno a muchos con los *Turnos*. Esto le permite a la agenda poseer la colección de citas para validar la disponibilidad horaria del profesional.
4. **Asociación Bidireccional (1 a Muchos):** Se optó por una relación bidireccional entre *Turno* y *Paciente*. El *Turno* conoce a qué *Paciente* pertenece, y a su vez, el *Paciente* guarda una lista interna con su historial de turnos. Esta decisión permite un acceso rápido y directo al historial médico desde el objeto *Paciente*, cumpliendo de forma nativa con el requerimiento de consultar los turnos asociados a una persona sin necesidad de recorrer repositorios externos en esta etapa temprana del desarrollo. Al momento de crear un turno, se asegura la sincronización agregando la instancia a ambas listas (*Agenda* y *Paciente*).
5. **Asociación con Enumerado:** La clase *Turno* se asocia fuertemente con *EstadoTurno* para definir su estado actual de forma tipada, segura y libre de errores de tipeo.
6. **Dependencia (Uso):** La *Recepcionista* depende de *Paciente* y *Turno* para ejecutar sus métodos de gestión, pero no los almacena como atributos propios.

COMPORTAMIENTO Y ENCAPSULAMIENTO

Todas las clases respetan el encapsulamiento (private con getters/setters) e implementan toString(), equals() y hashCode() basados en sus identificadores de negocio (DNI, Matrícula, Legajo).

Los métodos de lógica de negocio más relevantes incorporados son:

- En ***Paciente*** -> agregarTurno(Turno t) Permite añadir un turno a su historial privado sin exponer la lista completa.
- En ***Agenda*** -> estaDisponible(fecha, hora): boolean Delega la lógica de validación de horarios directamente a la colección que posee los datos.
- En ***Turno*** -> esFuturo(): boolean Compara la fecha/hora del turno con LocalDateTime.now() para facilitar futuras lógicas de confirmación o cancelación.
- En ***Recepcionista*** -> Métodos de Gestión (CRUD simulado) Se implementaron los métodos registrarPaciente, asignarTurno, modificarTurno y cancelarTurno. Para garantizar el realismo y la robustez del sistema, se incluyeron validaciones de dominio mediante métodos auxiliares privados:
    - **Validación Temporal:** Antes de asignar o modificar un turno, el método esFechaValida comprueba usando LocalDateTime.now().isAfter() que la fecha y hora solicitadas no pertenezcan al pasado.
    - **Validación de Disponibilidad:** Se delega a la *Agenda* la verificación de solapamientos horarios. Nota: Al asignar un turno válido, se garantiza la consistencia bidireccional guardando el turno tanto en la agenda como en el paciente. Se comprende que en la Entrega 2, bajo una arquitectura de capas, esta lógica será migrada a clases de tipo Servicio.

CONCLUSIONES

El modelo resultante es robusto, evita la duplicación de código mediante herencia, protege sus datos mediante encapsulamiento y delega responsabilidades de forma lógica. El código provisto en el paquete modelo, junto con la clase Main de prueba, demuestra que el sistema es capaz de simular el flujo completo de registro y asignación requerido para la primera etapa del proyecto.